

Week 5 Tutorial Notes

▼ Agenda

Q1 → simple programming language task

Q3 → setof integer

Q4 → harder programming language task

Q7,8,9,10 → beers db

Q12 → SQL and PLpgSQL functions

Q14 - 17 → Great PLpgSQL functions

Pre-Tute

▼ Assignment 1 due 11:59PM this Friday

- Please **don't** answer PLpgSQL questions using 'RETURN QUERY'
- vxdb02 is capable of handling 100 or so PostgreSQL servers simultaneously, so **running any code will take much longer on Friday**
- Autotest your work **on vxdb02** before submitting.
- "A reminder: there are no yeasts in the ass1.dump data, but **there will be yeasts in the unseen database used for marking**. If you want to check whether your Q8 function works with yeast, simply add a new ingredients(...) tuple of type yeast into the database, attached to some beer and then run Q8 on that beer."
- No Quiz 🎉🎉

▼ Help sessions

Week	Mon	Tue	Wed	Thu	Fri
4	12 pm - 2 pm	1 pm - 3 pm	2 pm - 4 pm	2 pm - 4 pm	1 pm - 3 pm
	On campus	On campus	On campus	On campus	On campus
	CSE Help K17	H13 Lawrence East	CSE Help K17	CSE Help K17	Mathews 309
	G05	1043	G05	G05	Rushik
	Kenneth Li	Kenneth Li	Xinzhang Chen	Henry Li	Sriramoju
5	12 pm - 2 pm	1 pm - 3 pm	2 pm - 4 pm	2 pm - 4 pm	1 pm - 3 pm
	On campus	On campus	On campus	On campus	On campus
	CSE Help K17	H13 Lawrence East	CSE Help K17	CSE Help K17	Mathews 309
	G05	1043	G05	G05	Rushik
	Kenneth Li	Rushik Sriramoju	Sooyoung Moon	Henry Li	Sriramoju
	Sooyoung Moon	Xinzhang Chen	Xinzhang Chen	Sooyoung Moon	Xinzhang Chen

SQL Views

```
CREATE VIEW view_name AS
SELECT
    *
FROM
    table_name
WHERE
    (conditions)
;
```

SQL Functions

```
CREATE OR REPLACE FUNCTION func_name(paramType1, paramType2,
...) RETURNS returnType
AS $$
    -- single SQL query
$$ LANGUAGE SQL;
```

Key characteristics

- A single SQL statement with no RETURN → just outputs the result of the query.
- If no named parameters → need to use positional parameter notation: \$1, \$2, \$3, ...
- Can only return what an SQL query can return (atomic value, single row, table)

PLpgSQL Functions

```
CREATE OR REPLACE FUNCTION func_name(param1 Type1, param2 Type2, ...) RETURNS returnType
AS $$
DECLARE
    -- declare variables here
    var1 varType;
    var2 varType := defaultValue;
    ...
BEGIN
    -- programming logic goes here
END;
$$ LANGUAGE PLpgSQL;
```

Key characteristics:

- Allows actual programming logic
- Parameters are always named
- `found` is a very helpful keyword for finding if a query had an output or not
- Requires RETURN

▼ Normal RETURNS

```
-- To print all suppliers who supplies all parts of a given colour (returning text)
CREATE OR REPLACE SuppliesAllParts(_colour text) RETURN S text
```

```

AS $$
DECLARE
    _sid    text;
    _ret    text := '';
BEGIN
    -- loop through tuples of the sql query
    for _sid in
        select
            S.sid
        from
            Suppliers S
        where not exists (
            (select P.pid from Parts P where P.colour =
            _colour)
            except
            (select C.pid from Catalog C where C.sid =
            S.sid)
        )
    loop
        _ret := _ret || _sid || e'\n'; -- use e'\n' for
multi-line output
    end loop;

    return _ret;
END;
$$ LANGUAGE plpgsql;

```

▼ RETURNS setof

```

-- If we wanted to do the same thing for a function ret
returning setof text
CREATE OR REPLACE SuppliesAllParts(_colour text) RETURN
S setof text
AS $$
DECLARE

```

```

    _sid      text;
BEGIN
    for _sid in
        select
            S.sid
        from
            Suppliers S
        where not exists (
            (select P.pid from Parts P where P.colour =
            _colour)
            except
            (select C.pid from Catalog C where C.sid =
            S.sid)
        )
    loop
        return next _sid;
    end loop;

    return;
END;
$$ LANGUAGE plpgsql;

```

- PERFORM vs SELECT

▼ PERFORM: For queries that you want to perform but don't want to store

```

CREATE OR REPLACE SuppliesPart(partial_name text) RETURN
NS text
AS $$
DECLARE
    _pid      integer;
BEGIN
    perform
        *
    from
        Parts

```

```

where
    pname ILIKE '%' || partial_name || '%' -- Case-insensitive partial string matching (search this up for more details)
;
-- typically you do this for when you want to check valid input.
-- when you do a perform or select query, it updates a keyword called 'found' which will be true if the query
-- outputted some tuples and false if no tuples
if not found then
    return 'No part matches';
end if;

...
END;
$$ LANGUAGE plpgsql;

```

▼ SELECT: For when you want to store variables

```

CREATE OR REPLACE SuppliesPart(partial_name text) RETURNS text
AS $$
DECLARE
    _pid integer;
BEGIN
    -- Note that below we assume the output of the query will be
    -- a single value, not multiple tuples (multiple matches)
    select
        pid -- which column of data
    into
        _pid -- which variable we're storing into

```

```

from
    Parts
where
    pname ILIKE '%' || partial_name || '%'
;
if not found then
    return 'No part matches';
end if;
-- now we can use _pid as a variable later in the c
ode.

...
END;
$$ LANGUAGE plpgsql;

```